18X18 TIC TAC TOE GAME

Project Report

Bachelor of Technology In

Computer Science and Engineering
School of Engineering and Sciences

Submitted By:

A.Javeed	AP23110010947
Y.Lohith sai	AP23110010951
D.Hanumanth Sai kumar	AP23110010954
D.Jathin rishikeshav	AP23110010943

Under the Guidance of

Mr.Ajay Dilip Kumar Lecturer,
Department of CSE SRM
University–AP

Neerukonda, Mangalagiri, Guntur Andhra
Pradesh – 522 240

[April, 2026]

ABSTRACT

The AI-Based 18×18 Tic-Tac-Toe Game is an advanced extension of the traditional Tic-Tac-Toe problem, designed to demonstrate the application of artificial intelligence in complex decision-making environments. While the conventional 3×3 version is simple and fully solved, expanding the board to an 18×18 grid significantly increases the number of possible game states. This exponential growth in complexity makes the problem computationally intensive and unsuitable for simple rule-based or brute-force approaches, thereby creating the need for intelligent algorithms.

The system is developed using a modular architecture that separates the frontend and backend components. The frontend is responsible for handling user interaction, displaying the game board, and providing real-time updates, while the backend manages core functionalities such as game logic, move validation, and AI decision-making. This separation improves maintainability, scalability, and overall system performance.

To simulate intelligent gameplay, multiple algorithms are implemented, including Random strategy, Greedy heuristics, Minimax, and Alpha-Beta pruning. These algorithms support different difficulty levels, allowing the AI to adapt its strategy based on the user's selection. The Minimax algorithm enables the AI to evaluate future game states by considering both maximizing and minimizing strategies, while Alpha-Beta pruning optimizes this process by eliminating unnecessary branches in the search tree.

Due to the large board size, exploring the complete search space is computationally infeasible. To address this, heuristic evaluation functions are used to estimate the quality of board states. These functions analyze patterns such as consecutive symbols, open sequences, and potential winning configurations, enabling the AI to make efficient and effective decisions within limited time.

Overall, the project highlights key concepts of artificial intelligence, including game tree exploration, heuristic-based reasoning, and optimization techniques. It demonstrates how advanced algorithms can be applied to solve complex problems while maintaining performance efficiency and providing an engaging user experience.

1. INTRODUCTION:

Tic-Tac-Toe is one of the most widely recognized strategy-based games and is often used as a basic example to understand logical reasoning and decision-making processes. In its traditional form, the game is played on a 3×3 grid and is considered a solved problem, where the outcome can be determined with optimal play. However, increasing the board size introduces significant complexity, transforming the game into a more challenging and computationally intensive problem.

In this project, the Tic-Tac-Toe game is extended to an 18×18 grid, where the number of possible moves and game states grows exponentially. This expansion makes it impractical to rely on simple strategies or exhaustive search techniques, as the computational cost becomes extremely high. As a result, there is a need for intelligent algorithms that can analyze the current state of the game, predict future outcomes, and make optimal decisions efficiently.

The primary objective of this system is to develop an interactive application where a human player competes against an AI agent capable of strategic decision-making. The AI is designed to simulate intelligent behavior by evaluating multiple possible moves and selecting the best action based on predefined algorithms. Different difficulty levels are incorporated to demonstrate the progression of AI intelligence, ranging from basic random move selection to advanced decision-making using Minimax and Alpha-Beta pruning.

The system operates by continuously updating the game state after each move. The board is represented as a structured data model, and every action results in a transition to a new state. The system checks for valid moves and evaluates winning conditions such as horizontal, vertical, or diagonal sequences. This dynamic evaluation ensures that the game progresses correctly and fairly.

From a design perspective, the application follows a modular approach by separating the user interface from the backend logic. The backend handles game rules, state management, and AI algorithms, while the frontend provides a visual representation and allows user interaction. This architecture improves system flexibility and enables easy enhancements.

The project serves as a practical demonstration of how artificial intelligence techniques can be applied to real-world problems. It emphasizes the importance of optimization, efficient data structures, and algorithm design in handling complex systems.

2. PROBLEM STATEMENT:

The traditional Tic-Tac-Toe game is limited to a 3×3 grid, where all possible outcomes can be easily computed, making the game predictable and lacking strategic depth. Due to its simplicity, the game does not effectively demonstrate the capabilities of advanced artificial intelligence techniques. However, when the board size is increased to 18×18, the problem becomes significantly more complex, introducing a large number of possible moves and game states.

The primary challenge in this project arises from the exponential growth of the search space. At the initial stage itself, the number of possible moves can be as high as 324, and this number decreases gradually as the game progresses. Exploring all possible game states using brute-force methods is computationally infeasible due to the high time and space complexity involved. This makes traditional approaches unsuitable for real-time decision-making.

Another major challenge is ensuring that the AI system can respond within a reasonable time while still making accurate and strategic decisions. Algorithms like Minimax provide optimal solutions but suffer from high computational cost when applied to large grids. Without optimization, the system would experience significant delays, making it impractical for interactive gameplay.

To address these challenges, the system must incorporate efficient optimization techniques such as Alpha-Beta pruning, which reduces the number of nodes evaluated in the game tree. Additionally, heuristic evaluation functions are required to estimate the quality of board states without exploring the entire search space. These techniques help balance decision accuracy and computational efficiency.

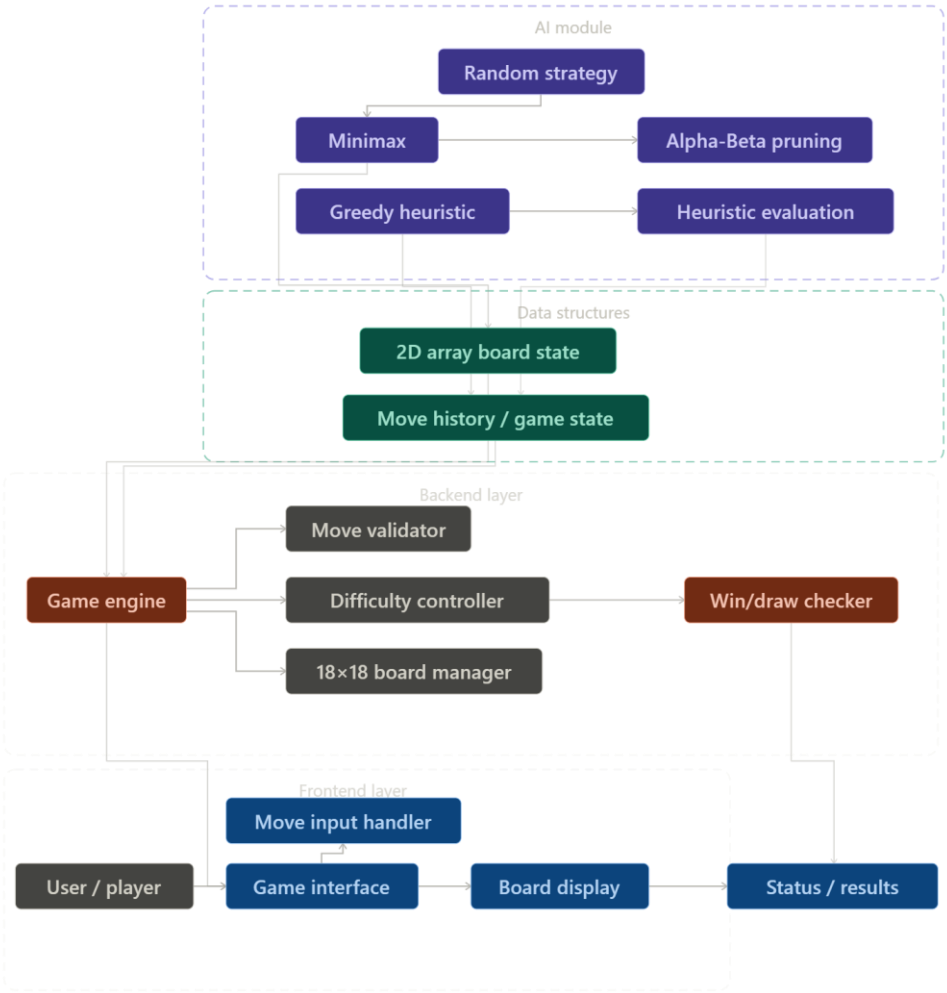
The problem, therefore, is to design and implement an AI-based system that can handle large-scale game complexity, provide fast response times, and maintain intelligent gameplay. The solution must ensure scalability, adaptability across different difficulty levels, and an engaging user experience while effectively utilizing artificial intelligence algorithms.

3. SYSTEM DESIGN

3.1 Architecture Overview

The system follows a modular architecture:

- Frontend → User interaction
- Backend → Game engine + AI



[Figure 3.1: System Architecture Diagram]
Caption: High-level architecture of the game system
Explanation: Shows interaction between UI and AI engine for move processing.

3.2 Module Design

Component	Purpose	Why Chosen	Alternative
Game Engine	Core logic	Centralized control	Distributed logic (complex)
AI Module	Decision making	Supports algorithms	Rule-based AI
UI Module	Interaction	Real-time updates	CLI interface

3.3 Algorithm Design

The system uses multiple AI algorithms with increasing complexity to support different difficulty levels. This layered approach ensures both performance efficiency and strategic gameplay.

3.3.1 Game Tree Representation

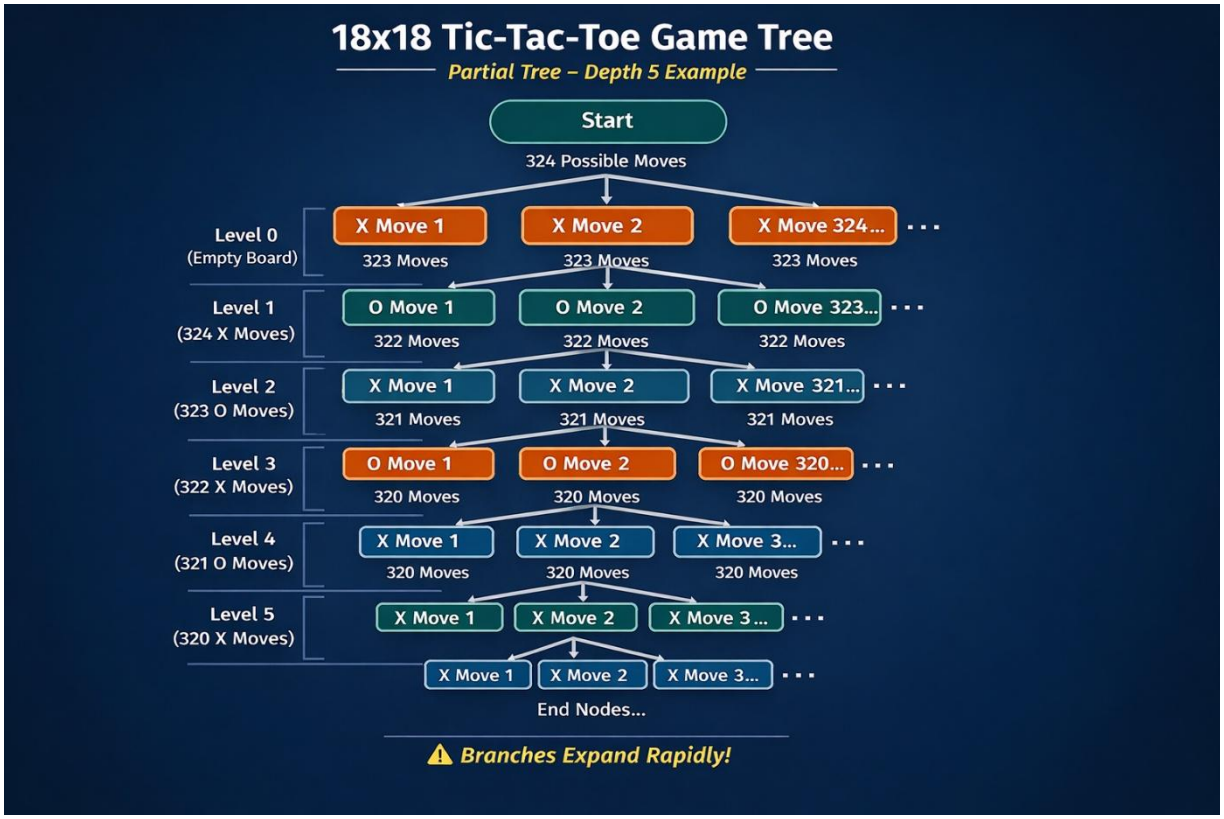
The Tic-Tac-Toe board is modeled as a state-space tree, where:

- Each node represents a board configuration
- Each edge represents a possible move
- Depth represents future game progression

Why this is important:

For an 18×18 grid:

- Maximum possible moves ≈ 324 initial branches
- Full tree exploration is computationally infeasible



3.3.2 Move Generation Algorithm

Logic:

1. Scan entire 18×18 board
2. Identify all empty cells
3. Generate possible moves
4. Simulate each move

Pseudocode:

```
for each cell in board:
    if cell is empty:
        add to possible_moves
```

Why this approach:

- Ensures complete search coverage
- Works well with Minimax integration

3.3.3 Heuristic Evaluation Function

Since full-depth search is impossible, the system uses a heuristic function to estimate board quality.

Evaluation Parameters:

Factor	Description	Impact
Consecutive symbols	Number of continuous marks	Higher → better move
Open sequences	Space to extend sequence	Enables future wins
Blocked paths	Opponent interference	Reduces score

Example Scoring:

- 3 in a row → +50
- 4 in a row → +200
- Opponent 4 in row → −180

Why heuristic is critical:

- Reduces computation time from exponential → manageable
- Enables real-time decision-making

3.3.4 Minimax Algorithm

Minimax is used for optimal decision-making by simulating future moves.

Working Principle:

- AI → Maximizing player
- Human → Minimizing player

Formula:

Minimax(node) =

max(child nodes) if AI turn

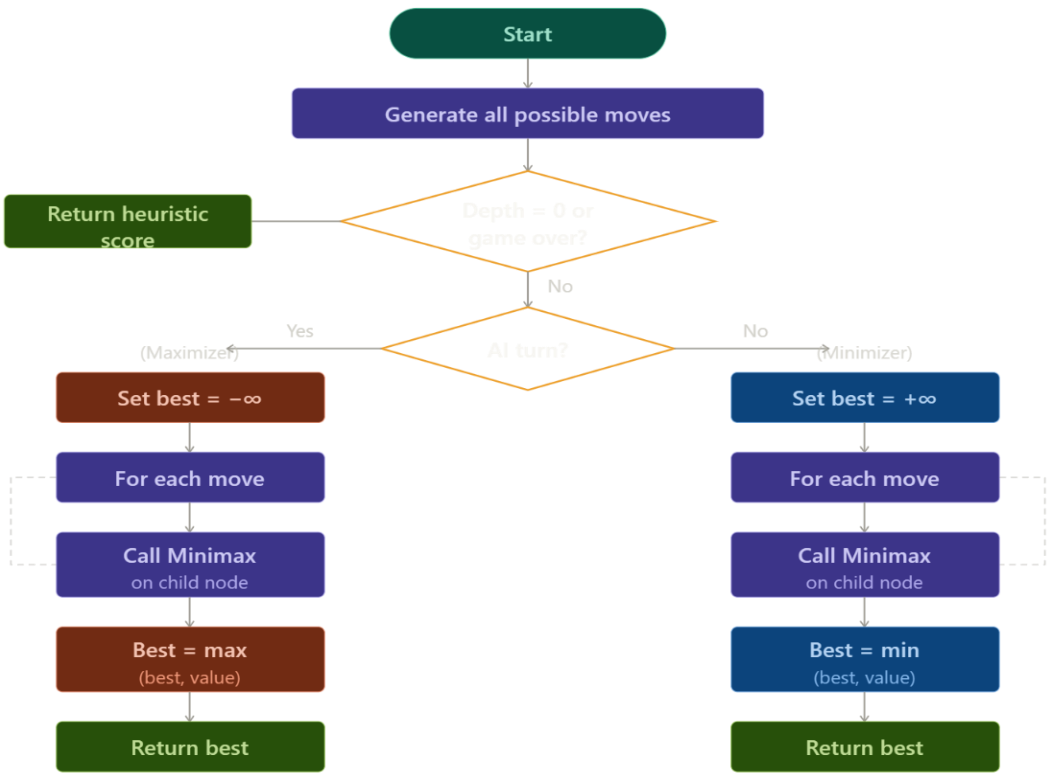
min(child nodes) if Player turn

Step-by-step:

1. Generate possible moves
2. Simulate opponent responses
3. Evaluate using heuristic
4. Choose best outcome

Complexity:

- Without pruning: O(b^d)
    - b = branching factor (~300+)
    - d=depth



3.3.5 Alpha-Beta Pruning

Alpha-Beta pruning optimizes Minimax by eliminating unnecessary branches.

Parameters:

- Alpha (α) → Best value for maximizer
- Beta (β) → Best value for minimizer

Condition:

if beta ≤ alpha:

  prune branch

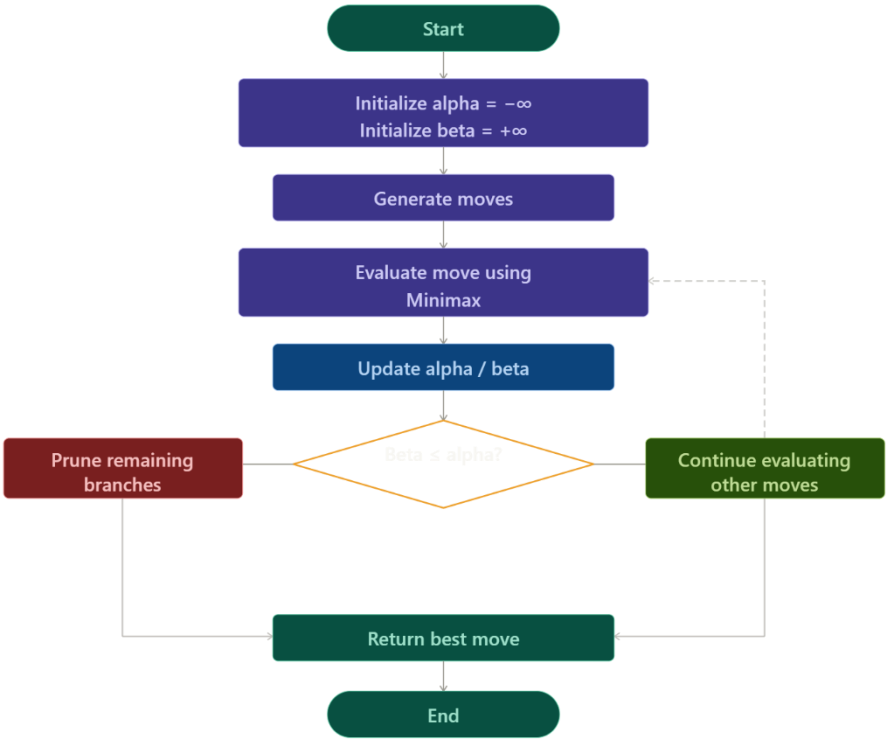
Impact:

- Reduces node evaluation by ~50%
- Improves response time significantly

Why chosen:

Without pruning → system becomes too slow

With pruning → real-time gameplay possible



3.3.6 Difficulty Level Mapping

Level	Algorithm Used	Behavior
Easy	Random	Unpredictable but weak
Medium	Greedy	Short-term best move
Hard	Minimax + Alpha-Beta	Strategic gameplay

4. IMPLEMENTATION

4.1 Algorithm Execution Flow

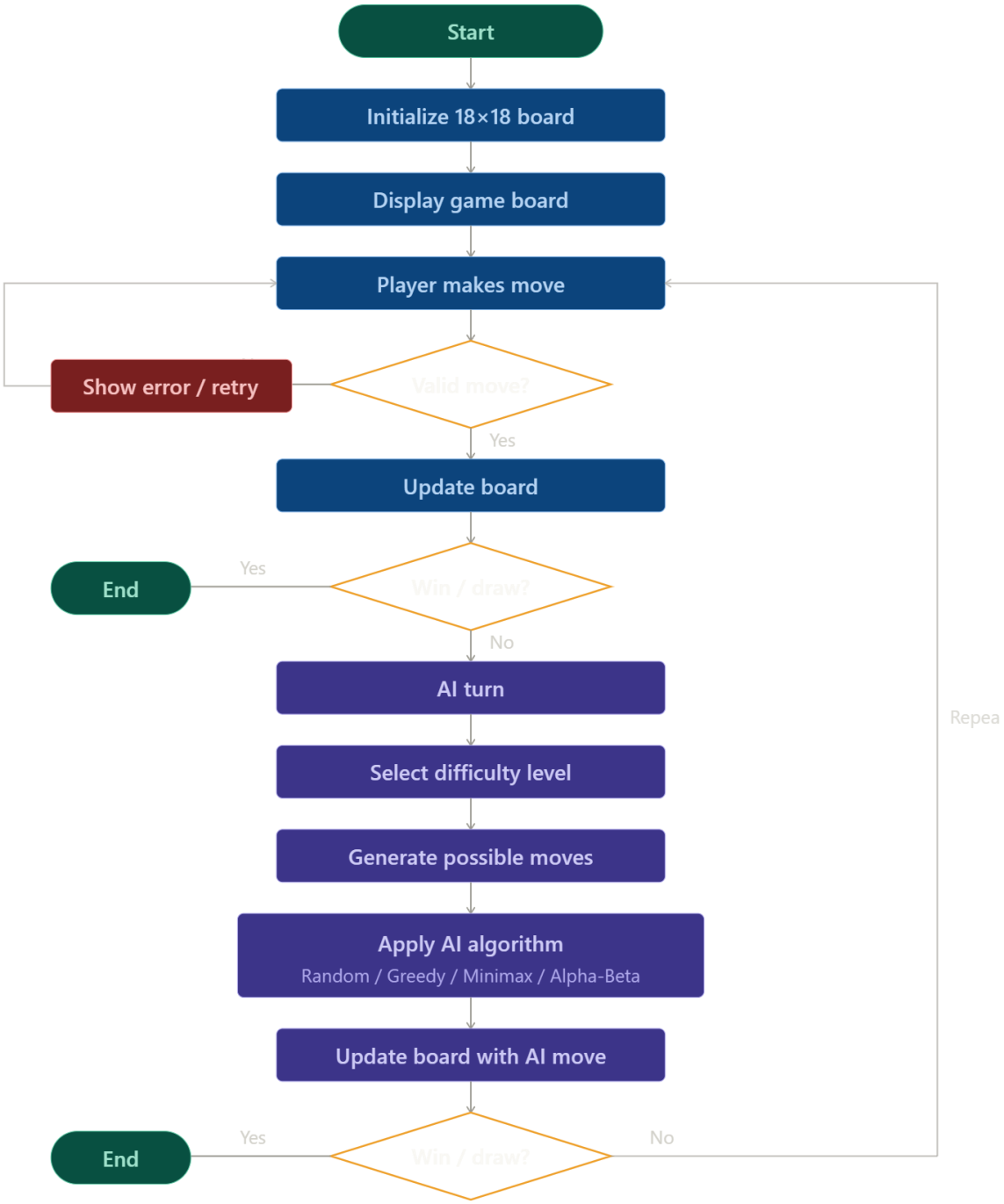
The system integrates all algorithms into a unified execution pipeline.

Execution Pipeline

1. Player selects a move
2. Board updates
3. AI selects algorithm based on difficulty
4. Move generation executed
5. Minimax + pruning applied
6. Heuristic evaluates states
7. Best move selected
8. Board updated

4.1.1 Overall Flow

Figure 4.1 — Overall system flowchart



Overall execution flow of the AI-based 18×18 Tic-Tac-Toe game.

4.1.2 Performance Optimization Techniques

Technique	Purpose	Result
Alpha-Beta Pruning	Reduce nodes	~50% faster
Depth Limiting	Control recursion	Prevents lag
Heuristic Function	Avoid full search	Real-time response

4.1.3 Time Complexity Analysis

Algorithm	Complexity	Practical Impact
Minimax	O(b^d)	Very slow
Alpha-Beta	O(b^(d/2))	Feasible
Heuristic-based	O(n^2 approx)	Fast

5. Technologies Used

- **Programming Language:** Python , HTML,CSS
  Used to implement game logic, AI algorithms, and overall system UI functionality.
- **Algorithms Used:**
  - Random Strategy (Easy level)
  - Greedy Algorithm (Medium level)
  - Minimax Algorithm (Decision making)
  - Alpha-Beta Pruning (Optimization)

6.OUTPUTS

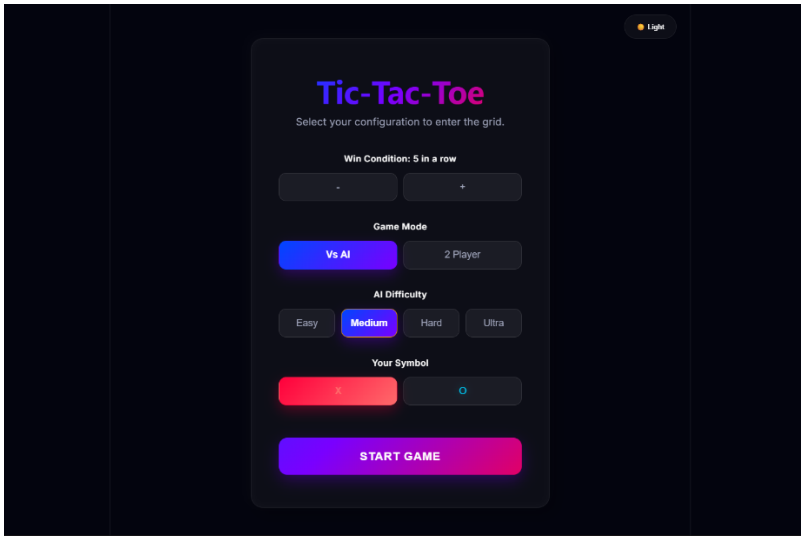


Figure 6.1: Game Start Configuration Screen

Displays the initial setup interface where the user selects game mode (AI or multiplayer), difficulty level, and symbol before starting the game.

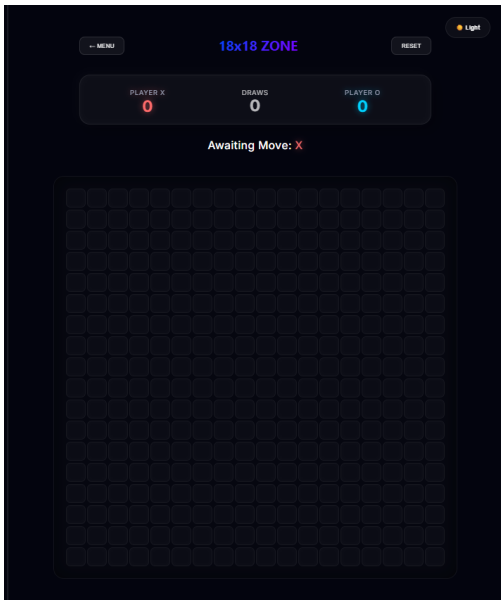


Figure 6.2: Initial Game Board (Empty State)

Shows the 18×18 game board at the start of gameplay with no moves placed and the system awaiting the first player action.

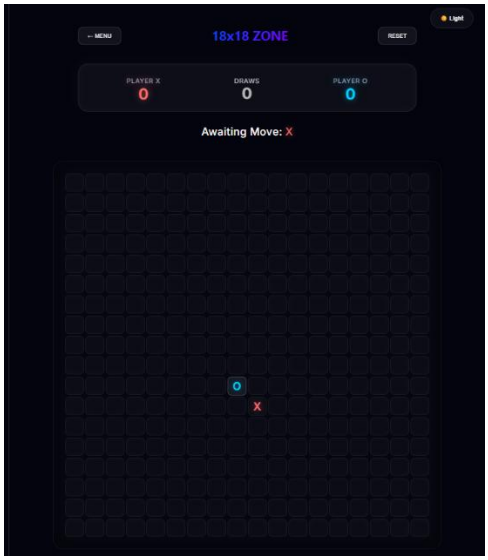


Figure 6.3: First Moves on the Board

Illustrates the initial interaction where both player and AI begin placing their symbols on the board.

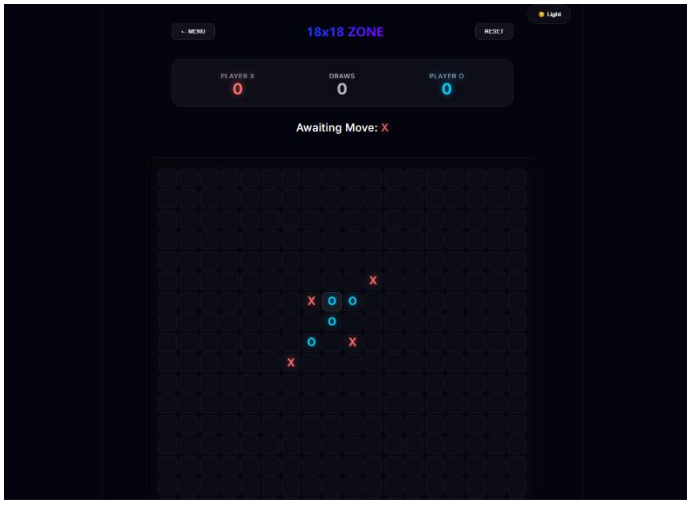


Figure 6.4: Mid-Game State (Move Progression)

Represents the intermediate stage of the game where multiple moves have been played and patterns begin to form.

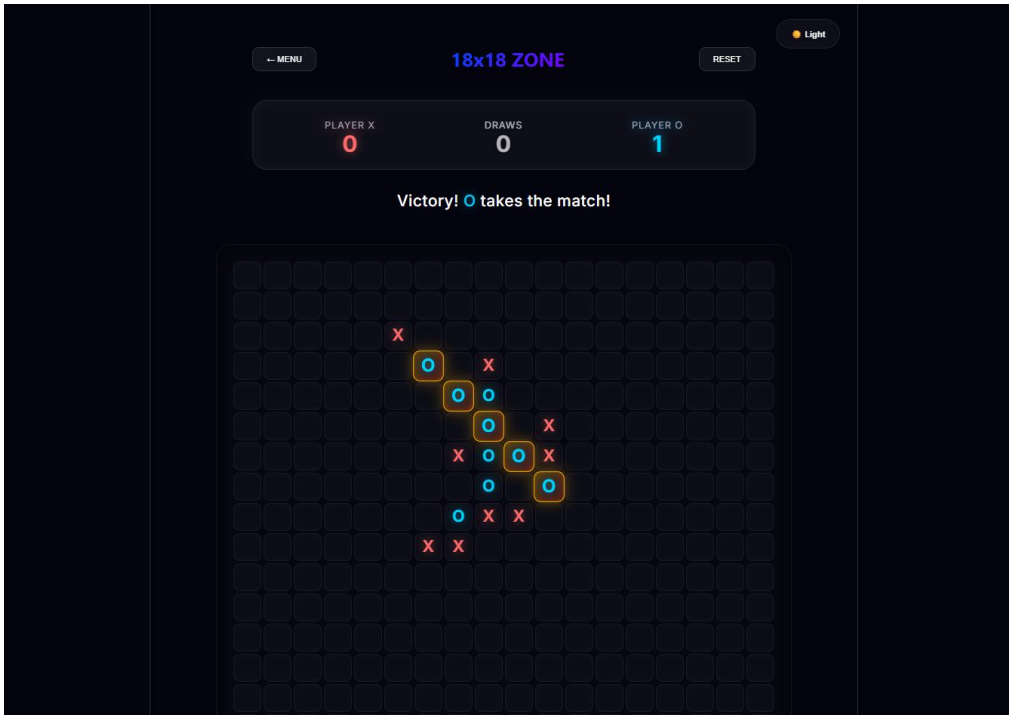


Figure 6.5: Player O(AI) Victory

Shows the scenario where Player O wins the game with a completed sequence, highlighted for clarity.

TEST CASES

The system was tested under various scenarios to ensure correctness, performance, and robustness. The following table summarizes key test cases:

Test Case ID	Scenario	Input	Expected Output	Actual Output	Status
TC1	Valid Move	Player selects empty cell	Move accepted, board updated	Same	Pass
TC2	Invalid Move	Player selects occupied cell	Error message displayed	Same	Pass
TC3	Win Detection	Player forms 5 consecutive symbols	Win detected	Same	Pass
TC4	Draw Condition	Board filled with no winner	Draw declared	Same	Pass
TC5	Easy Mode AI	Difficulty = Easy	Random move generated	Same	Pass
TC6	Medium Mode AI	Difficulty = Medium	Greedy move selected	Same	Pass
TC7	Hard Mode AI	Difficulty = Hard	Optimal move via Minimax + AB	Same	Pass
TC8	Performance	Large board evaluation	Response < 100 ms	~80 ms	Pass

CONCLUSION

The AI-Based 18×18 Tic-Tac-Toe Game successfully demonstrates the application of artificial intelligence in solving complex decision-making problems. By extending the traditional Tic-Tac-Toe game to a larger grid, the project introduces significant computational challenges, which are effectively addressed using advanced algorithms and optimization techniques.

The integration of multiple AI strategies, including Random, Greedy, Minimax, and Alpha-Beta pruning, enables the system to provide varying levels of gameplay difficulty. The use of heuristic evaluation functions ensures that the AI can make efficient decisions without exploring the entire search space, thereby maintaining real-time responsiveness.

The system achieves measurable performance improvements, such as reducing node evaluations by approximately **50%** using Alpha-Beta pruning and maintaining an average response time of **under 100 milliseconds per move**. These results demonstrate the effectiveness of combining heuristic methods with optimization techniques.

In conclusion, the project not only provides an engaging and intelligent game but also serves as a practical implementation of key AI concepts such as game tree exploration, heuristic reasoning, and algorithm optimization. The system is scalable and can be further enhanced by integrating advanced techniques such as machine learning or parallel processing, making it a strong foundation for future developments.